Assignment 1: Analysis of SVD for Fine-Tuning LLMs

Big Data Analytics Techniques and Applications (Fall 2026) | Student ID: 315581033 (邱泓崴)
Code (public GitHub repository): https://github.com/Rewrite-StarrySky/BigData2026-Assignment1

Setup. PiSSA [1] fine-tuning of Llama-3.2-1B on the first 25k MetaMathQA samples (24,975 used), all 112 target matrices (q/k/v/o/gate/up/down in 16 layers), 1 epoch, learning rate 2e-5, FP16, effective batch 128 (196 updates), max 256 new tokens; evaluation on the first 1,000 test questions each of GSM8K and MATH (accuracy = correct / 1,000); Kaggle 2×T4, seed 42. Every number below comes from a single run. SE = $\sqrt{(p(1-p)/1000)}$; z = difference / SE of the difference (unpaired). "Unparsed" = no number in a GSM8K output, or no answer marker ("answer …", "####", `\boxed{}`) in a MATH output (counted as wrong). Parts beyond the assignment are marked **extra experiment**.

Question 1 (1%): Accuracy vs. target rank

(a) Report the accuracy of the original Llama-3.2-1B on GSM8K and MATH. (b) Fine-tune with target rank 16, 64 and 256 (component = default) and report the accuracy. (c) Provide observations and explanations on the performance trends with different target ranks for both datasets.

Accuracy	Original LLM	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
GSM8K	0.035	0.157	0.204	0.274
MATH	0.001	0.038	0.046	0.054

(c) Observations and explanations

- **Fine-tuning helps substantially.** GSM8K rises from 0.035 to 0.157–0.274 (4.5×–7.8×) and MATH from 0.001 to 0.038–0.054. On MATH the original model rarely ends with a parseable final answer (956/1,000 outputs have no answer marker, capping its accuracy at 4.4%); MetaMathQA responses end with "The answer is: …" and after fine-tuning only 456–543 lack one, so part of the gain is learning to finish with a parseable answer. On GSM8K only 15 original outputs contain no number (0 after fine-tuning) yet only 35 are correct: the problem there is wrong numbers, not missing ones (generations were not inspected).
- **Accuracy increases monotonically with rank on both datasets.** GSM8K: +4.7 pts (16→64; z = 2.7) and +7.0 pts (64→256; z = 3.7); MATH: +0.8 (z = 0.9) and +0.8 pts (z = 0.8). A larger rank gives more trainable parameters (11.3M / 45.1M / 180.4M = 0.9% / 3.5% / 12.7% of all parameters incl. adapters) and a larger principal subspace to adapt (Q4: ranks 16 / 64 / 256 retain 15–35% / 36–51% / 70–76% of the energy of q_proj in layers 0 / 7 / 15; the r = 256 values come from the extra experiment), so more of the fine-tuning signal can be absorbed (a plausible mechanism; the runs do not isolate it). The last-update training loss also falls (0.455 → 0.414 → 0.375; mean of only 15 examples, weak evidence) and the GSM8K gain has not saturated at r = 256.
- **GSM8K improves far more than MATH.** Even among MATH outputs that contain an answer marker only 8.3–9.9% are correct, so MATH is much harder for this 1B model. The MATH gain from r = 16 to 256 splits into two similar factors: outputs with a marker 457 → 544 and accuracy among them 8.3% → 9.9%. 456–543 of 1,000 MATH outputs still have no marker (shrinking with rank: 543 → 473 → 456); a possible reason, *not verified* (generations were not saved), is solutions cut off by the 256-token limit.
- **Caveat.** One seed and n = 1,000 give a binomial standard error of ≈1.2–1.4 pts (GSM8K) and ≈0.6–0.7 pts (MATH). The GSM8K trend is clear (16→256: +11.7 pts, z = 6.4); adjacent-rank MATH gaps (0.8 pts) are only suggestive, 16→256 (1.6 pts) has z = 1.7.

Question 2 (1%): Training time vs. target rank

(a) Following Question 1, record the training time (minutes) for target ranks 16, 64 and 256. (b) Provide observations and explanations on the training-time trends with different target ranks.

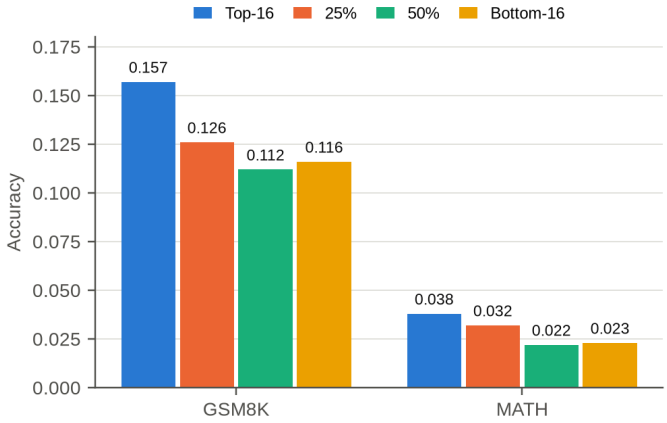
	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
Training Time (min.)	66.12	71.43	96.90
Relative to RANK = 16	1.00×	1.08×	1.47×
Trainable parameters	11.3M	45.1M	180.4M

(b) Observations and explanations

- **Time grows with rank, but sub-linearly.** Quadrupling the rank (16→64) costs only +5.3 min (+8%); a 16× rank (16→256) costs +30.8 min (+47%), although the trainable parameters grow 16×. That is 0.11 / 0.13 min per rank unit (16→64 / 64→256); the line through r = 16 and 256 extrapolates to ≈64 min at r = 0 (arithmetic on three runs, not a fitted model).
- **Explanation.** Each PiSSA layer computes $y = xW_{res}^T + (xB^T)A^T$. The frozen W_{res} matmuls (forward, plus the backward pass that propagates activation gradients) cost the same at every rank, while the adapter adds $r(d_{in}+d_{out})$ multiply-adds per token per layer in the forward pass (about twice that in the backward pass), i.e. 1.2% / 4.6% / 18.5% of the 0.97B dense projection weights at r = 16 / 64 / 256. Counting these matmuls predicts only ≈26% more time at r = 256 than at r = 16, versus the measured +47%. Not profiled; possible causes of the excess: the adapter branch runs in FP32 outside autocast while the frozen path runs in FP16 (T4: 8.1 TFLOPS FP32 vs. 65 TFLOPS FP16/FP32 mixed precision [2]), skinny matmuls, and optimizer / gradient handling of 16× more trainable parameters.
- **Notes.** The one-time SVD initialisation of the 112 matrices takes 143–151 s (2.4–2.5 min) in all six runs and is reported separately from the training time. The four rank-16 runs of Q3 train for 65.8–66.3 min, so cost depends on the rank, not on which singular values are selected.

Question 3 (1%): Which singular values to fine-tune (rank = 16)

(a) For each dataset, perform SVD and fine-tune with four ranges of singular values at target rank 16 — Top-16 (component = default), 16 consecutive values starting at the 25% position, starting at the 50% position, and the Bottom-16 — and report the accuracy in bar charts. (b) Provide observations and explanations on the performance trends for both datasets.



The four singular-value ranges (rank = 16)

	Top-16	25%	50%	Bottom-16
GSM8K	0.157	0.126	0.112	0.116
MATH	0.038	0.032	0.022	0.023
Train loss†	0.455	0.495	0.497	0.507
Time (min.)	66.1	65.8	66.3	66.1

† Loss of the last update only (mean of its 15 examples; noisy). The 25% / 50% ranges start at value n/4 + 1 / n/2 + 1 of the n singular values (n = 2,048; 512 for k_proj and v_proj).

(b) Observations and explanations

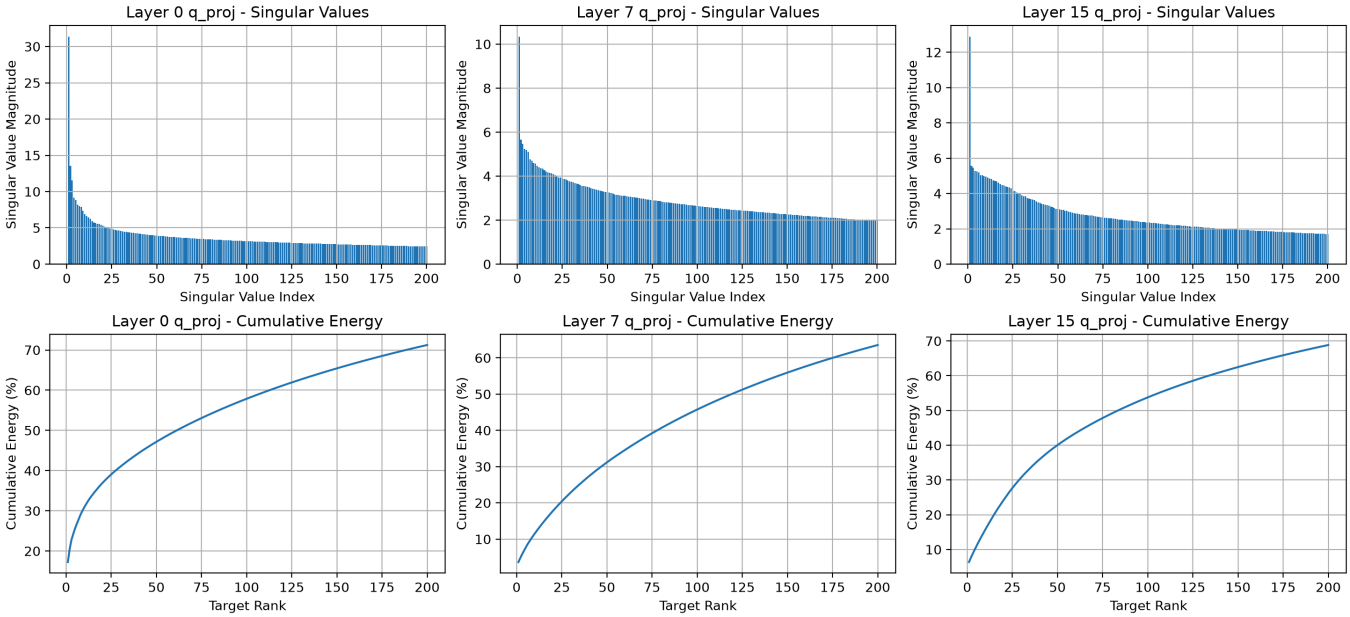
- **Top-16 is best on both datasets, but clearly so only on GSM8K:** GSM8K 0.157 vs. 0.112–0.126 (+3.1 to +4.5 pts, +25% to +40% relative; z = 2.0–3.0) and MATH 0.038 vs. 0.022–0.032 (+0.6 to +1.6 pts; z = 0.7 against the 25% range, 2.1 and 2.0 against 50% and Bottom). The other three ranges lie within 1.4 pts (GSM8K) and 1.0 pt (MATH) of each other (z ≤ 1.4) and are not ordered by position (25% > Bottom > 50%), so we do not read a trend among them.
- **Why the principal components win.** All four runs start from the same function (W_{res}+AB = W; max. relative reconstruction error ≤ 2.1e-04), so the difference comes only from which directions are trained. The PiSSA paper argues that principal singular vectors are "the directions in which the matrix W has the most significant stretching or impact", so tuning them lets the model "fit the training data faster and better", whereas LoRA's noise-and-zero initialisation gives gradients that "could be small and uninformative for a long time" [1, Sec. 1]. The other ranges consist of smaller singular values (in q_proj their indices are ≥ 513, so $\sigma \le \sigma_{200}$ = 1.7–2.4, versus σ_1 = 10–31), so $A = U_r S_r^{1/2}$ and $B = S_r^{1/2} V_r^T$ start closer to zero, somewhat like LoRA's zero initialisation (*our own reasoning:* a step changes W by ΔA·B + A·ΔB, which scales with $\sigma^{1/2}$). The paper's own comparison of principal, medium and minor components (three 7B models, MetaMathQA) also finds lower training loss and higher GSM8K and MATH accuracy for the principal ones [1, App. F]. The last-update loss (0.455 vs. 0.495–0.507) agrees but is a 15-example average.
- **Every variant still beats the original model** (GSM8K 0.035, MATH 0.001), even Bottom-16, so part of the gain does not depend on the subspace: MATH outputs with an answer marker rise from 44 to 457 (Top-16) and 385–411 (other ranges), suggesting that all variants pick up the answer format of the training data (an interpretation; generations were not inspected).

Question 4 (2%): SVD of q_proj in the original Llama-3.2-1B (no fine-tuning)

(a) Perform SVD on the q_proj weights of layers 0, 7, 15. (b) Report, for each layer, the minimum target rank that retains at least 50% cumulative energy (energy of rank r = $\Sigma_{i\le r}\sigma_i^2 / \Sigma_{all}\sigma_i^2$). (c) Visualise the first 200 singular values per layer: cumulative energy (line chart) and singular-value magnitude (bar chart). (d) Provide your observations.

	Layer 0	Layer 7	Layer 15
Minimum target rank for ≥ 50% energy	**62**	**120**	**84**
Cumulative energy at r = 16 / 64 / 200 (%)	34.8 / 50.6 / 71.2	15.4 / 35.9 / 63.5	21.0 / 44.7 / 68.8
Cumulative energy at r = 256 (%)‡	76.4	70.1	74.3
σ_1 (share of total energy)‡	31.4 (17.2%)	10.3 (3.7%)	12.9 (6.4%)

‡ From the extra experiment below (same weights, `svdvals`); it agrees with the Q4 notebook wherever both report a quantity.



Top row: singular-value magnitudes (bar charts); bottom row: cumulative energy (line charts). First 200 singular values of each layer's q_proj (2048 × 2048), as output by the Q4 notebook.

(d) Observations

- **q_proj is only weakly low-rank.** The spectrum has a steep head (σ_1 = 31.4 / 10.3 / 12.9, i.e. 12.9× / 5.2× / 7.6× σ_{200}) but then decays slowly: 50% of the energy needs 62 / 120 / 84 singular values (3.0% / 5.9% / 4.1% of 2,048), and the first 200 hold only 63.5–71.2%. The smallest singular values (7×10^{-6} to 8×10^{-5}) are tiny but not zero; in float32 they are only accurate to about $\varepsilon\cdot\sigma_1 \approx 1\text{–}4\times10^{-6}$, so we do not interpret them. The cumulative-energy curves are concave and still rising at rank 200.
- **Energy concentration differs by layer and is not monotonic in depth.** Layer 0 is the most concentrated (σ_1 alone holds 17.2%; min. rank 62), layer 7 the flattest of the three (σ_1 3.7%; min. rank 120) and layer 15 lies in between (6.4%; 84).
- **Implications.** (i) One global rank keeps different fractions per layer (r = 64 keeps 51% / 36% / 45% in layers 0 / 7 / 15), so layer 7 needs a larger rank than layers 0 and 15 for the same fidelity. (ii) Ranks 16 / 64 / 256 cover only ≈15–35% / 36–51% / 70–76% of q_proj's energy in these layers, in line with Q1 where accuracy kept improving up to r = 256. (iii) Only the Top-16 range of Q3 comes from the steep head of the spectrum (in q_proj the 25% / 50% / Bottom ranges start at index 513 / 1,025 / 2,033). These are indicative only: Q4 looks at q_proj of three layers, Q1/Q3 adapt all 7 matrix types in 16 layers.

Extra experiment (not required by the assignment): q_proj of all 16 layers

To check whether layer 7 is typical, the same SVD (`torch.linalg.svdvals`, float32) was run on the q_proj weight of every layer (`extra_q4_all_layers.ipynb`); its results for layers 0 / 7 / 15 agree with Q4 (same minimum ranks, energies within 10^{-3} pt). Minimum target rank for ≥ 50% energy:

Layer	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
Min. rank, ≥ 50%	62	174	184	172	162	144	139	120	118	114	118	144	146	152	74	84

- Layers 1–13 all need ≥ 114 singular values (maximum: layer 2, 184), whereas layers 0, 14 and 15 need only 62, 74 and 84. So the flattest layer is layer 2, not layer 7, and layer 7 (120) is typical of the interior layers (114–184). The PiSSA authors suspect that "middle layers in a model tend to have more even eigenvalue distributions due to redundancy and insufficient training" [1, App. G]; our measurement is consistent with this for q_proj but does not test the cause.
- Over all 16 layers, ranks 16 / 64 / 256 cover 11.7–34.8% / 27.9–50.6% / 59.4–76.5% of the q_proj energy; even rank 256 leaves 24–41% in the frozen residual.

[1] F. Meng, Z. Wang, M. Zhang. PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models. NeurIPS 2024 (arXiv:2404.02948). [2] NVIDIA T4 specifications, nvidia.com/en-us/data-center/tesla-t4. All numbers are taken from the executed notebooks in the repository (bigdata-hw1-q1/q3/q4.ipynb, extra_q4_all_layers.ipynb).